

Introduction to the Laboratory ARMKit with MCXC143VFT

Juraj Sýkora

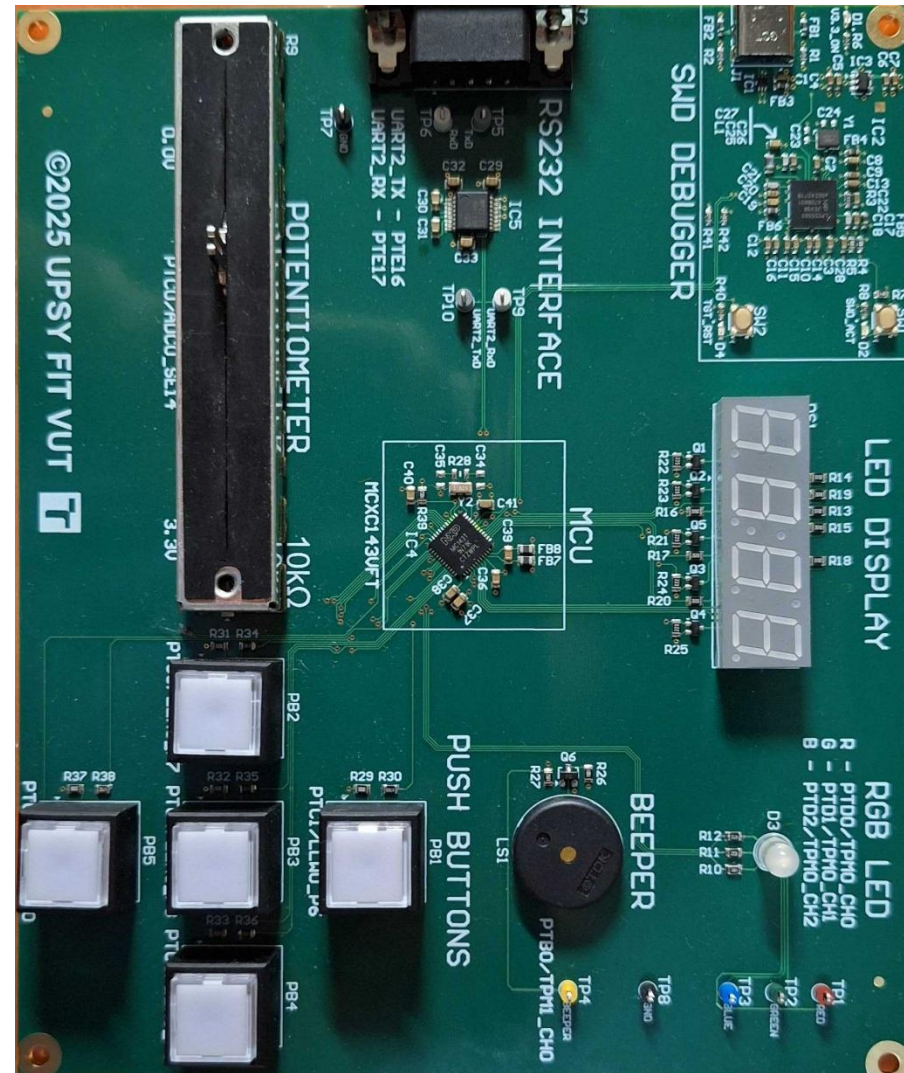
Brno University of Technology, Faculty of Information Technology
Božetěchova 1/2, 612 66 Brno, Czech republic
xsykorj00@fit.vutbr.cz



20.05.2026

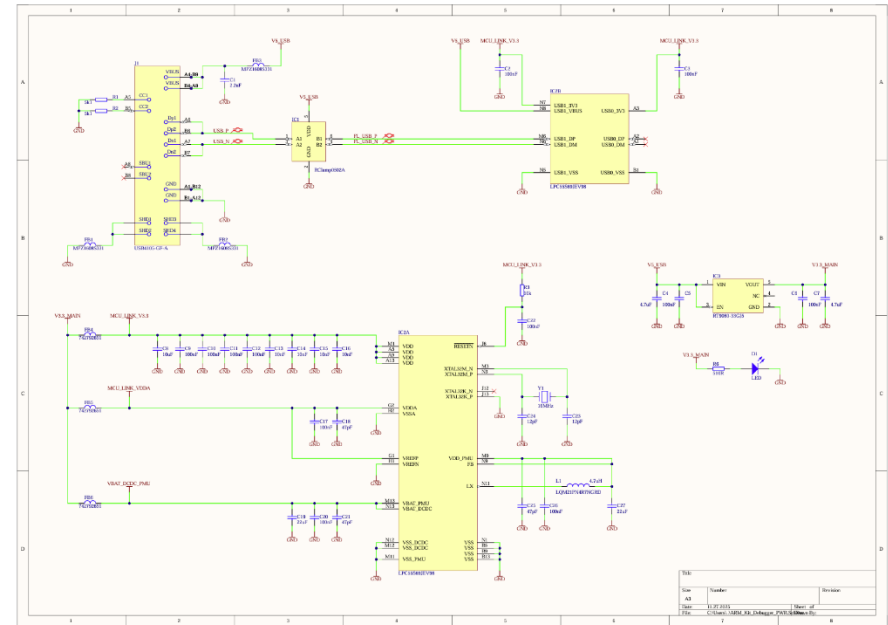
- 32-bit ARM MCU laboratory kit
- Schematic: debug interface and power supply
- Schematic: target MCU MCXC143VFT and peripherals
- MCUXpresso IDE– project creation
- MCUXpresso IDE– build, flash and debug

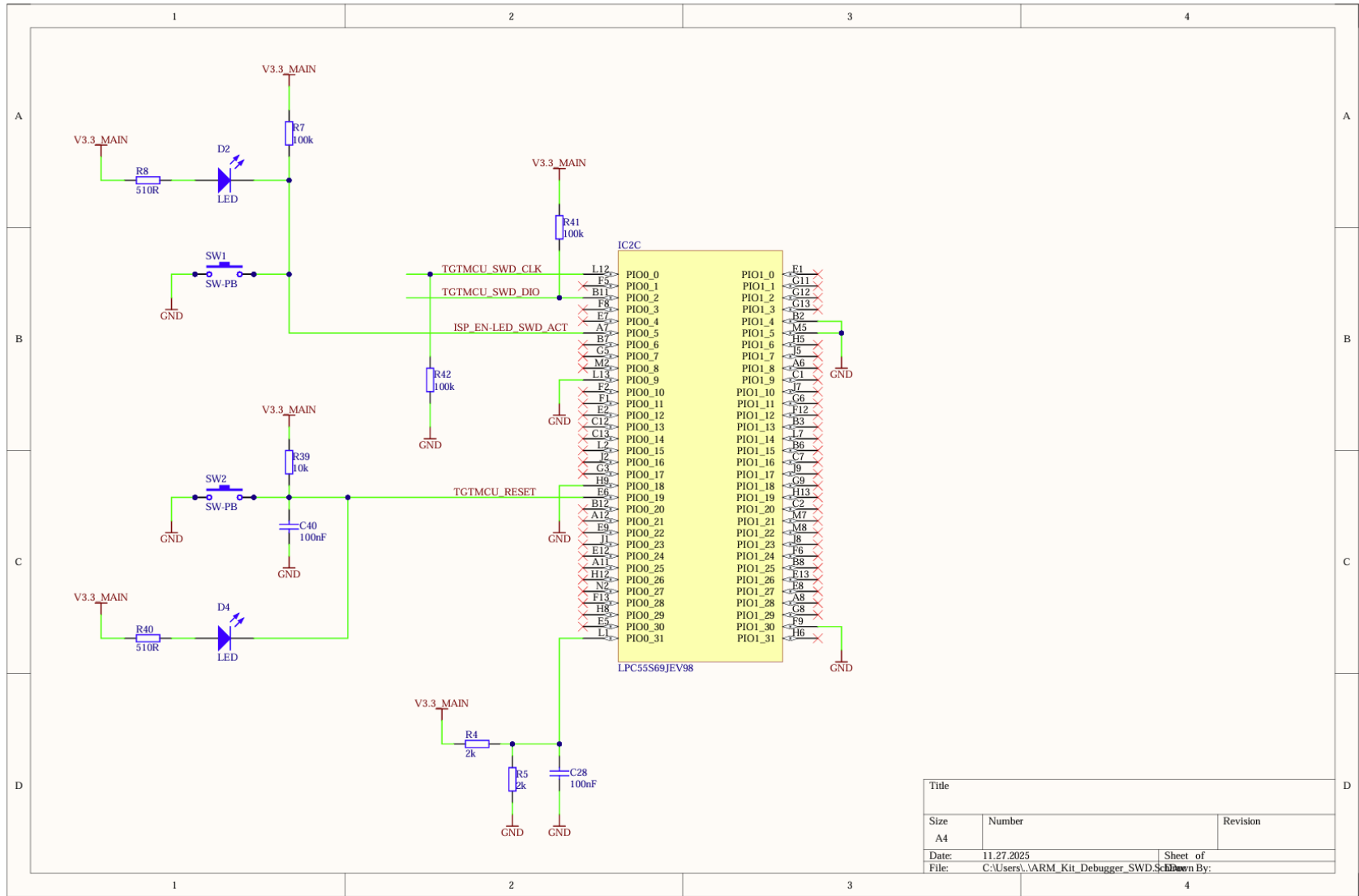
- Kits are located in laboratory **L306**
- Target MCU: NXP **MCXC143VFT**
- ARM **Cortex-M0+** core, up to **48MHz**
- 256kB Flash, 32kB SRAM
- 48-pin LQFP package
- On-board debugger: **MCU-Link v3** (NXP LPC55S69JEV98)
- **USB-C** connector– power + CMSIS-DAP/SWD debug
- 3.3V main supply via **RT9080-33** LDO



Key components:

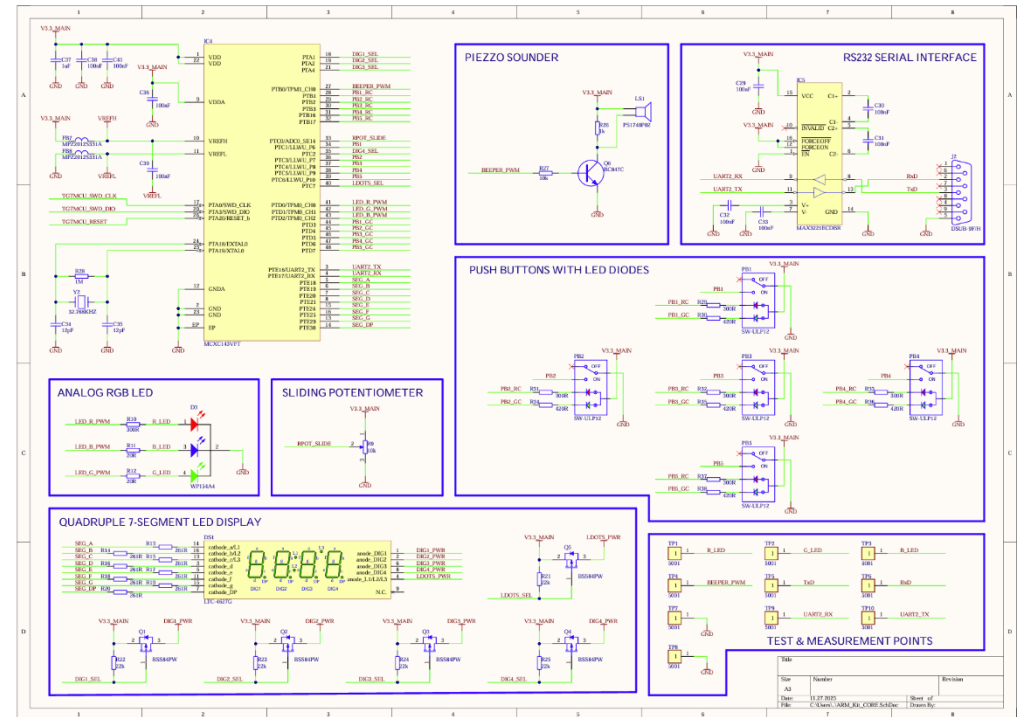
- **USB4105-GF-A** – USB-C receptacle
- **LPC55S69JEV98** – MCU-Link v3 chip
- **RT9080-33GJ5** – 3.3V LDO regulator
- **MPZ1608S331** – ferrite bead EMI filters
- **Y1** – 16MHz crystal for MCU-Link





Key pin assignments:

- **PTA0/SWD CLK, PTA3/SWD DIO, PTA20/RESET b – SWD debug interface**
- **PTE16/UART2 TX, PTE17/UART2 RX – RS-232 via MAX3221E**
- **PTD0-2/TPM0 CH0-2 – RGB LED PWM**
- **PTB0/TPM1 CH0 – buzzer PWM**
- **PTC0/ADC0 SE14 – sliding potentiometer**
- **VDDA/VREFH/VREFL – analog supply and reference**



Push buttons PB1 – PB5 (SW-ULP12):

- PBx_RC/PBx_GC – GPIO

and LED anode

Analog RGB LED (D3 WP154A4):

- LED R/G/B PWM

driven by TPM0_CH0–2

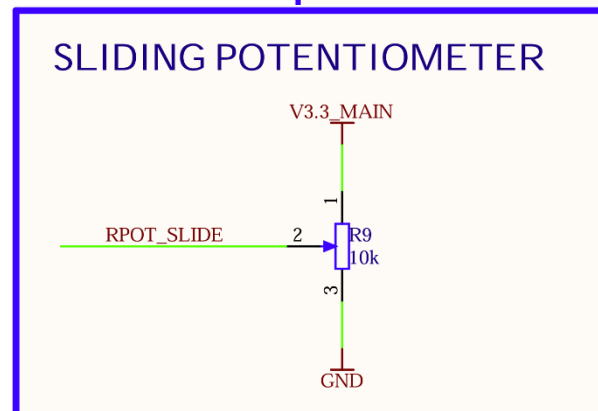
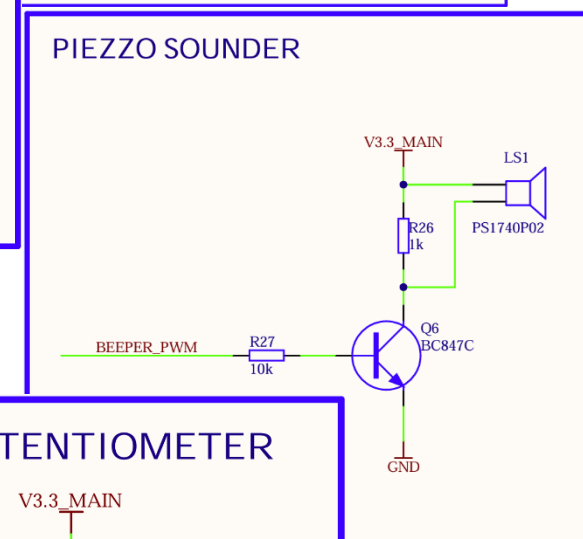
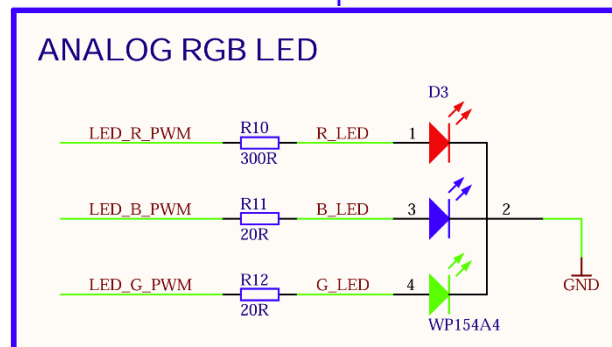
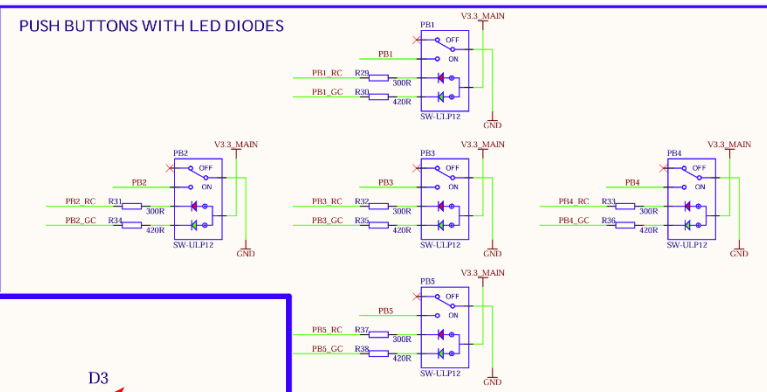
- R10=300Ω, R11/R12=20Ω

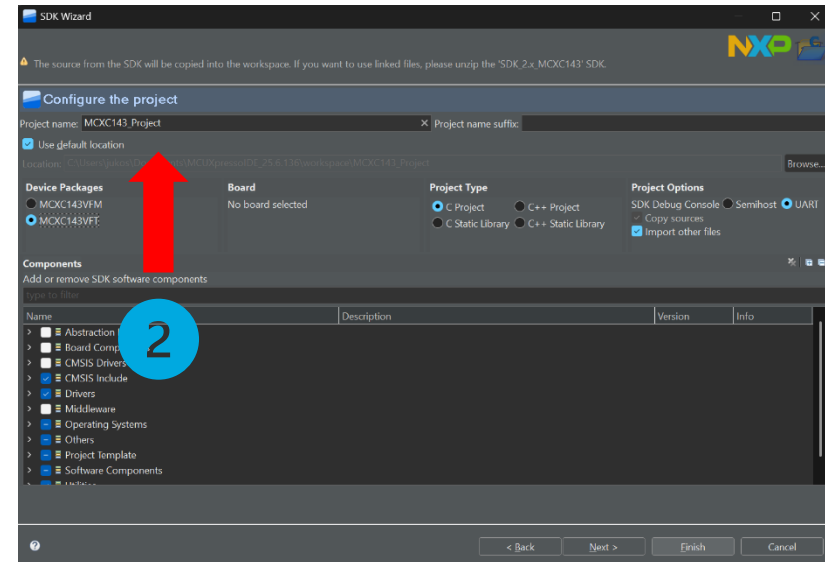
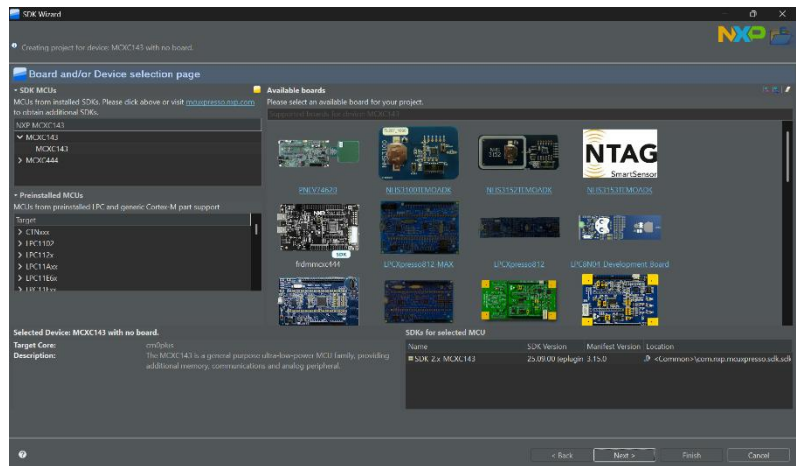
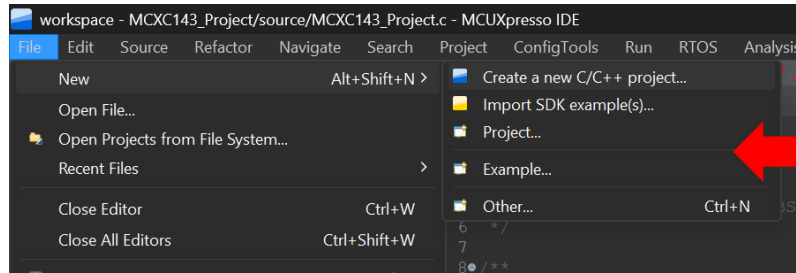
Piezo buzzer (LS1):

- NPNdriver Q6 (BC847C)
- Signal: BEEPER_PWM on PTB0

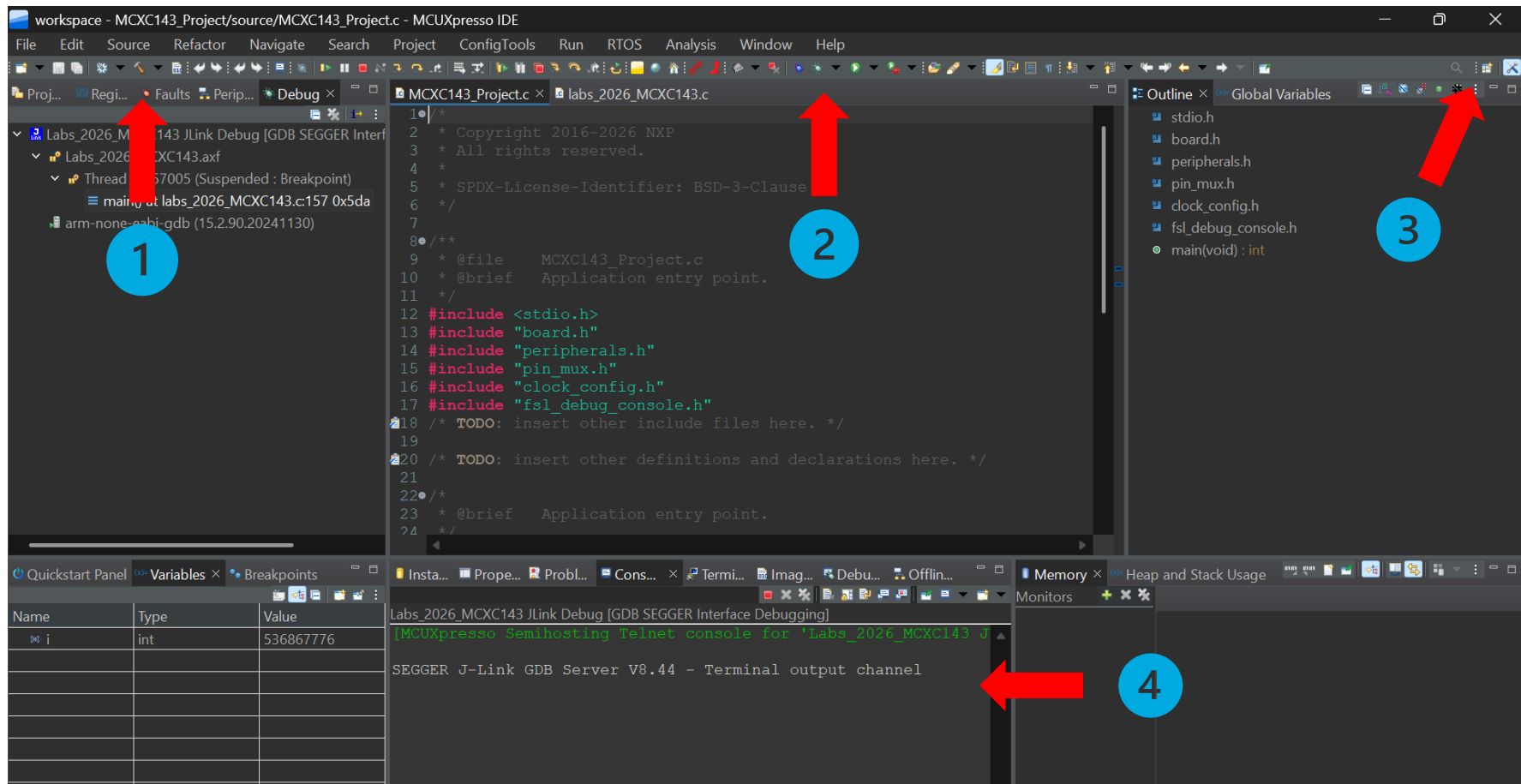
Sliding potentiometer:

- RPOT SLIDE → ADC0 SE14





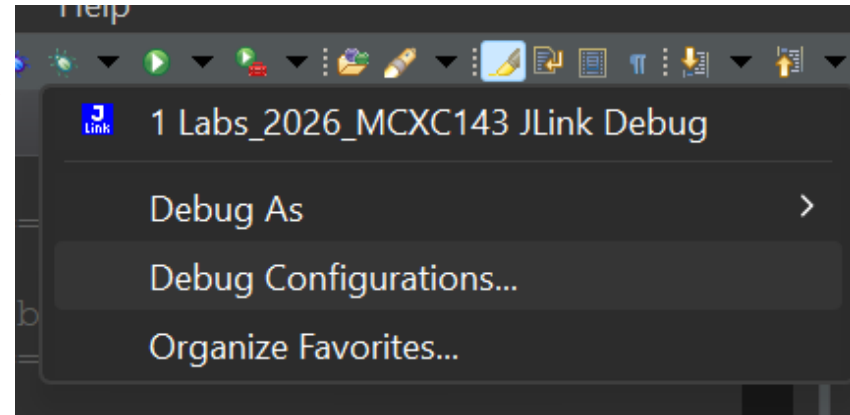
- 1) **File** → **New** → **Create a new C/C++ project**; Select **NXP MCUXpresso SDK Project**; Choose **SDK for MCXC143VFT** (download from mcuxpresso.nxp.com if not yet installed)
- 2) Enter a **project name**; set the workspace, e.g. **D:/IMP/** or **P:/IMP Labs/**
- 3) Select language **C**; leave the default linker script and click **Finish**



- 1) **Build** (hammer icon) – compiles sources and links the binary
- 2) **Debug** (bug icon) – starts an interactive debug session
- 3) **Perspective switcher** – toggle between C/C++ (edit) and Debug views
- 4) **Console panel** – build output; check here for errors
- 5) **Variables/Registers/ Memory** – inspect MCU state at runtime **next to console panel**

- 1) Arrow next to **Debug** → **Debug Configurations**
- 2) Expand **GDB SEGGER Interface Debugging**; select or create the project entry

1



Main tab – verify:

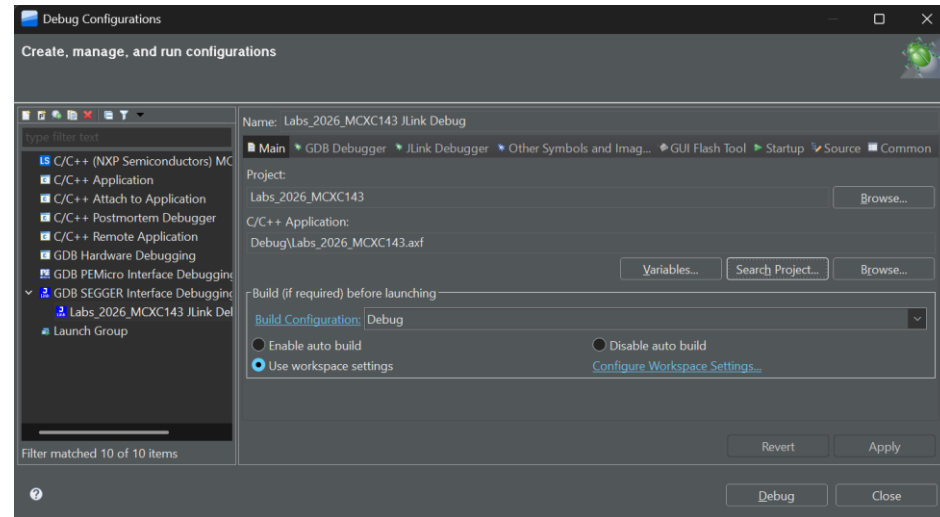
- **Project** – your project name
- **C/C++ Application** path

3. JLink Debugger tab verify:

- Device: **MCXC143**
- Target Interface: **SWD**
- JLink Interface: **USB**

- 4) Click **Debug** and resume execution: **F8** or green **Resume** button

2



- 1) Arrow next to **Debug** → **Debug Configurations**
- 2) Expand **GDB SEGGER Interface Debugging**; select or create the project entry

3

Main tab – verify:

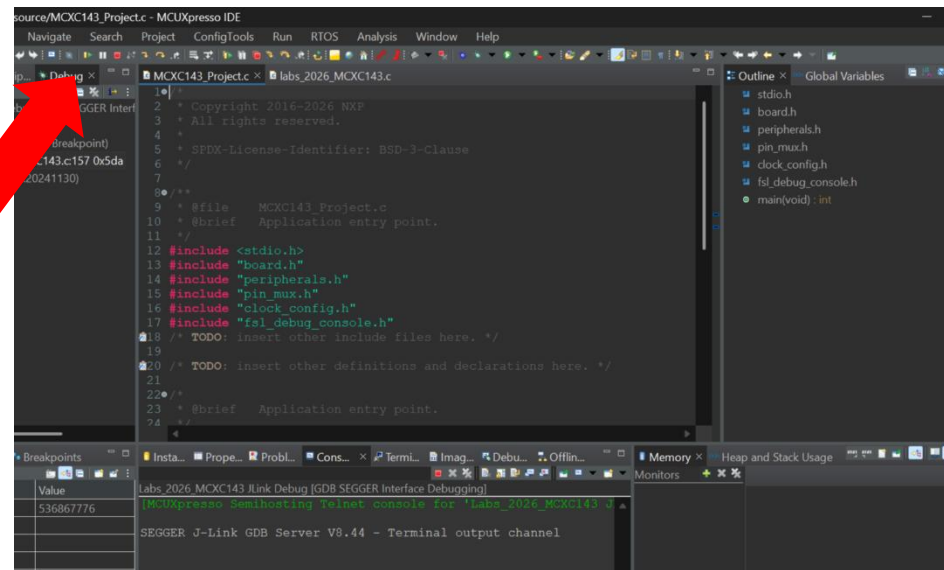
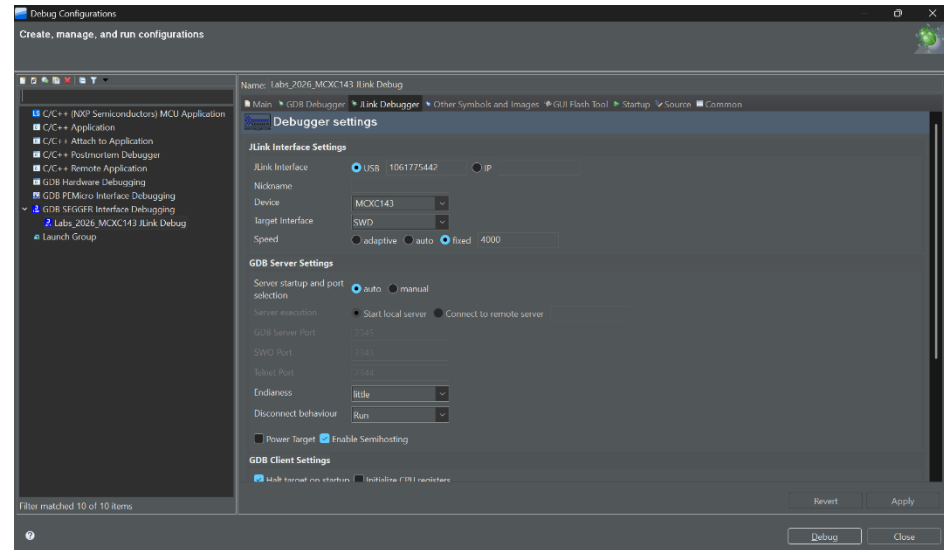
- **Project** – your project name
- **C/C++ Application** path

3. **JLink Debugger** tab verify:

- Device: **MCXC143**
- Target Interface: **SWD**
- JLink Interface: **USB**

- 4) Click **Debug** and resume execution: **F8** or green **Resume** button

4





- 1) **Resume (F8)** – starts or continues execution on the MCU
- 2) **Suspend** – pauses execution; inspect variables, registers, memory
- 3) **Terminate (Ctrl+F2)** – ends the debug session. Confirm the entry in the **Debug** view turns **grey**. A red entry means the session is still active – **cannot be restarted!**
- 4) After terminating, click the **C/C++** perspective to return to the editor

Stuck debug session

If the debug thread hangs, right-click `<terminated>` in the **Debug** view and choose **Terminate and Remove** or click **Clean Up Debug**.

```
#include "MCXC143.h"

static void delay(volatile uint32_t n) {
    while (n--);
}

int main(void) {
    /* Disable watchdog explicitly */
    SIM->COPC = SIM_COPC_COPT(0);
    /* Enable clock gate for Port B */
    SIM->SCGC5 |= SIM_SCGC5_PORTB_MASK;
    /* PTB0 -> GPIO output (MUX = 1) */
    PORTB->PCR[0] = PORT_PCR_MUX(1);
    GPIOB->PDDR |= (1u << 0);
    GPIOB->PCOR = (1u << 0); /* start LOW */
    for (;;) {
        GPIOB->PSOR = (1u << 0); /* PTB0 high */
        delay(50000u);
        GPIOB->PCOR = (1u << 0); /* PTB0 low */
        delay(50000u);
    }
    return 0;
}
```

Notes:

- Header `MCXC143.h` selected for MCXC143
- Use atomic `PSOR/PCOR` instead of read-modify write on `PDOR`
- Watchdog disabled explicitly via `SIM->COPC`
- **BEEPER PWM** is wired to **PTB0** on the kit (see schematic ARM Kit CORE)

Override in your application code:

```
void DMA0_IRQHandler(void);
void DMA1_IRQHandler(void);
void DMA2_IRQHandler(void);
void DMA3_IRQHandler(void);
void FTFE_IRQHandler(void);          /* Flash done */
void LVD_LVW_IRQHandler(void);
void LLWU_IRQHandler(void);
void I2C0_IRQHandler(void);
void SPI0_IRQHandler(void);
void UART2_IRQHandler(void);        /* RS-232 */
void ADC0_IRQHandler(void);
void CMP0_IRQHandler(void);
void TPM0_IRQHandler(void);         /* RGB PWM */
void TPM1_IRQHandler(void);         /* Buzzer PWM */
void RTC_IRQHandler(void);
void PIT_IRQHandler(void);
void LPTMR0_IRQHandler(void);
void PORTA_IRQHandler(void);
void PORTB_IRQHandler(void);
void PORTC_PORTD_IRQHandler(void); /* C and D */
void PORTE_IRQHandler(void);
```

Thank You For Your Attention!